SKEIN ENGINE

VOLUME 07

PHYSICS, ANIMATION, AUDIO, INPUT AND NAVIGATION

Jolt integration, body components, character control, soft bodies, animation, audio, input, navigation and lightweight AI utilities.

DOCUMENT CODE	**SKEIN-IMP-002-V07**
VERSION	**0.1**
STATUS	**DRAFT CONSTRUCTION REFERENCE**
PLANNED PAGES	**190**
CHAPTERS	**15**
DATE	**30 July 2026**

IMPLEMENTATION MANUAL / PRINT SET

Current architecture source: SKEIN-SPEC-001 v0.4 and SKEIN-IMP-002-MASTER v0.1

07.01 Simulation scheduling and fixed timestep

Purpose and boundary: tick order

This page defines the purpose and boundary for Simulation scheduling and fixed timestep, with particular attention to tick order. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Tick order is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with substeps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tick order; distinguish required behavior from illustrative implementation detail.
- Keep substeps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationSchedulingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationSchedulingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationSchedulingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and tick order.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Architecture: substeps

This page defines the architecture for Simulation scheduling and fixed timestep, with particular attention to substeps. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Substeps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with interpolation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for substeps; distinguish required behavior from illustrative implementation detail.
- Keep interpolation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationSchedulingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and substeps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Data model: interpolation

This page defines the data model for Simulation scheduling and fixed timestep, with particular attention to interpolation. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Interpolation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pause is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for interpolation; distinguish required behavior from illustrative implementation detail.
- Keep pause behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationSchedulingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationSchedulingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationSchedulingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and interpolation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Public API: pause

This page defines the public api for Simulation scheduling and fixed timestep, with particular attention to pause. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Pause is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pause; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationSchedulingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and pause.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Ownership and lifetime: determinism

This page defines the ownership and lifetime for Simulation scheduling and fixed timestep, with particular attention to determinism. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tick order is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep tick order behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationSchedulingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationSchedulingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationSchedulingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Threading model: tick order

This page defines the threading model for Simulation scheduling and fixed timestep, with particular attention to tick order. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Tick order is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with substeps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tick order; distinguish required behavior from illustrative implementation detail.
- Keep substeps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationSchedulingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and tick order.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Memory strategy: substeps

This page defines the memory strategy for Simulation scheduling and fixed timestep, with particular attention to substeps. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Substeps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with interpolation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for substeps; distinguish required behavior from illustrative implementation detail.
- Keep interpolation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationSchedulingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationSchedulingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationSchedulingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and substeps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Failure handling: interpolation

This page defines the failure handling for Simulation scheduling and fixed timestep, with particular attention to interpolation. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Interpolation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with pause is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for interpolation; distinguish required behavior from illustrative implementation detail.
- Keep pause behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationSchedulingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and interpolation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Diagnostics: pause

This page defines the diagnostics for Simulation scheduling and fixed timestep, with particular attention to pause. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Pause is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for pause; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationSchedulingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationSchedulingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationSchedulingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and pause.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Implementation sequence: determinism

This page defines the implementation sequence for Simulation scheduling and fixed timestep, with particular attention to determinism. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tick order is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep tick order behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationSchedulingAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.01 Simulation scheduling and fixed timestep

Verification: tick order

This page defines the verification for Simulation scheduling and fixed timestep, with particular attention to tick order. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Tick order is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with substeps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tick order; distinguish required behavior from illustrative implementation detail.
- Keep substeps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationSchedulingAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationSchedulingAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationSchedulingAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.01 and tick order.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Purpose and boundary: third-party isolation

This page defines the purpose and boundary for Jolt integration boundary, with particular attention to third-party isolation. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Third-party isolation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with allocators is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for third-party isolation; distinguish required behavior from illustrative implementation detail.
- Keep allocators behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and third-party isolation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Architecture: allocators

This page defines the architecture for Jolt integration boundary, with particular attention to allocators. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Allocators is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with jobs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for allocators; distinguish required behavior from illustrative implementation detail.
- Keep jobs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Jolt", "JoltIntegrationBoundary");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and allocators.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Data model: jobs

This page defines the data model for Jolt integration boundary, with particular attention to jobs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Jobs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with coordinate conversion is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for jobs; distinguish required behavior from illustrative implementation detail.
- Keep coordinate conversion behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and jobs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Public API: coordinate conversion

This page defines the public api for Jolt integration boundary, with particular attention to coordinate conversion. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Coordinate conversion is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for coordinate conversion; distinguish required behavior from illustrative implementation detail.
- Keep debug behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Jolt", "JoltIntegrationBoundary");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and coordinate conversion.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Ownership and lifetime: debug

This page defines the ownership and lifetime for Jolt integration boundary, with particular attention to debug. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Debug is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with third-party isolation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug; distinguish required behavior from illustrative implementation detail.
- Keep third-party isolation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and debug.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Threading model: third-party isolation

This page defines the threading model for Jolt integration boundary, with particular attention to third-party isolation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Third-party isolation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with allocators is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for third-party isolation; distinguish required behavior from illustrative implementation detail.
- Keep allocators behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Jolt", "JoltIntegrationBoundary");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and third-party isolation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Memory strategy: allocators

This page defines the memory strategy for Jolt integration boundary, with particular attention to allocators. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Allocators is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with jobs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for allocators; distinguish required behavior from illustrative implementation detail.
- Keep jobs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and allocators.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Failure handling: jobs

This page defines the failure handling for Jolt integration boundary, with particular attention to jobs. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Jobs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with coordinate conversion is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for jobs; distinguish required behavior from illustrative implementation detail.
- Keep coordinate conversion behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Jolt", "JoltIntegrationBoundary");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and jobs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Diagnostics: coordinate conversion

This page defines the diagnostics for Jolt integration boundary, with particular attention to coordinate conversion. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Coordinate conversion is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with debug is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for coordinate conversion; distinguish required behavior from illustrative implementation detail.
- Keep debug behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and coordinate conversion.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Implementation sequence: debug

This page defines the implementation sequence for Jolt integration boundary, with particular attention to debug. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Debug is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with third-party isolation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for debug; distinguish required behavior from illustrative implementation detail.
- Keep third-party isolation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Jolt", "JoltIntegrationBoundary");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and debug.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Verification: third-party isolation

This page defines the verification for Jolt integration boundary, with particular attention to third-party isolation. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Third-party isolation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with allocators is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for third-party isolation; distinguish required behavior from illustrative implementation detail.
- Keep allocators behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and third-party isolation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Completion gate: allocators

This page defines the completion gate for Jolt integration boundary, with particular attention to allocators. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Allocators is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with jobs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for allocators; distinguish required behavior from illustrative implementation detail.
- Keep jobs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Jolt", "JoltIntegrationBoundary");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and allocators.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.02 Jolt integration boundary

Purpose and boundary: jobs

This page defines the purpose and boundary for Jolt integration boundary, with particular attention to jobs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Jobs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with coordinate conversion is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for jobs; distinguish required behavior from illustrative implementation detail.
- Keep coordinate conversion behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class JoltIntegrationBoundaryService final
{
public:
    [[nodiscard]] Result<void> Initialise(const JoltIntegrationBoundaryConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    JoltIntegrationBoundaryState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.02 and jobs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Purpose and boundary: primitives

This page defines the purpose and boundary for Collision shapes and materials, with particular attention to primitives. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Primitives is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with convex is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for primitives; distinguish required behavior from illustrative implementation detail.
- Keep convex behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and primitives.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Architecture: convex

This page defines the architecture for Collision shapes and materials, with particular attention to convex. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Convex is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with mesh is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for convex; distinguish required behavior from illustrative implementation detail.
- Keep mesh behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Collision", "CollisionShapesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and convex.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Data model: mesh

This page defines the data model for Collision shapes and materials, with particular attention to mesh. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Mesh is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heightfield is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for mesh; distinguish required behavior from illustrative implementation detail.
- Keep heightfield behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and mesh.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Public API: heightfield

This page defines the public api for Collision shapes and materials, with particular attention to heightfield. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Heightfield is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compound is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heightfield; distinguish required behavior from illustrative implementation detail.
- Keep compound behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Collision", "CollisionShapesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and heightfield.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Ownership and lifetime: compound

This page defines the ownership and lifetime for Collision shapes and materials, with particular attention to compound. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Compound is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cooking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compound; distinguish required behavior from illustrative implementation detail.
- Keep cooking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and compound.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Threading model: cooking

This page defines the threading model for Collision shapes and materials, with particular attention to cooking. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Cooking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with primitives is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cooking; distinguish required behavior from illustrative implementation detail.
- Keep primitives behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Collision", "CollisionShapesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and cooking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Memory strategy: primitives

This page defines the memory strategy for Collision shapes and materials, with particular attention to primitives. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Primitives is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with convex is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for primitives; distinguish required behavior from illustrative implementation detail.
- Keep convex behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and primitives.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Failure handling: convex

This page defines the failure handling for Collision shapes and materials, with particular attention to convex. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Convex is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with mesh is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for convex; distinguish required behavior from illustrative implementation detail.
- Keep mesh behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Collision", "CollisionShapesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and convex.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Diagnostics: mesh

This page defines the diagnostics for Collision shapes and materials, with particular attention to mesh. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Mesh is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with heightfield is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for mesh; distinguish required behavior from illustrative implementation detail.
- Keep heightfield behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and mesh.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Implementation sequence: heightfield

This page defines the implementation sequence for Collision shapes and materials, with particular attention to heightfield. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Heightfield is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compound is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for heightfield; distinguish required behavior from illustrative implementation detail.
- Keep compound behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Collision", "CollisionShapesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and heightfield.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Verification: compound

This page defines the verification for Collision shapes and materials, with particular attention to compound. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Compound is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with cooking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compound; distinguish required behavior from illustrative implementation detail.
- Keep cooking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and compound.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Completion gate: cooking

This page defines the completion gate for Collision shapes and materials, with particular attention to cooking. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Cooking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with primitives is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for cooking; distinguish required behavior from illustrative implementation detail.
- Keep primitives behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Collision", "CollisionShapesAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and cooking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.03 Collision shapes and materials

Purpose and boundary: primitives

This page defines the purpose and boundary for Collision shapes and materials, with particular attention to primitives. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Primitives is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with convex is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for primitives; distinguish required behavior from illustrative implementation detail.
- Keep convex behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class CollisionShapesAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const CollisionShapesAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    CollisionShapesAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.03 and primitives.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Purpose and boundary: components

This page defines the purpose and boundary for StaticBody3D and RigidBody3D, with particular attention to components. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Components is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with mass is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for components; distinguish required behavior from illustrative implementation detail.
- Keep mass behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and components.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Architecture: mass

This page defines the architecture for StaticBody3D and RigidBody3D, with particular attention to mass. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Mass is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sleeping is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for mass; distinguish required behavior from illustrative implementation detail.
- Keep sleeping behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and mass.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Data model: sleeping

This page defines the data model for StaticBody3D and RigidBody3D, with particular attention to sleeping. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Sleeping is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with CCD is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sleeping; distinguish required behavior from illustrative implementation detail.
- Keep CCD behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and sleeping.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Public API: CCD

This page defines the public api for StaticBody3D and RigidBody3D, with particular attention to CCD. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Ccd is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for CCD; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and CCD.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Ownership and lifetime: events

This page defines the ownership and lifetime for StaticBody3D and RigidBody3D, with particular attention to events. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with teleports is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep teleports behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Threading model: teleports

This page defines the threading model for StaticBody3D and RigidBody3D, with particular attention to teleports. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Teleports is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with components is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for teleports; distinguish required behavior from illustrative implementation detail.
- Keep components behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and teleports.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Memory strategy: components

This page defines the memory strategy for StaticBody3D and RigidBody3D, with particular attention to components. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Components is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with mass is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for components; distinguish required behavior from illustrative implementation detail.
- Keep mass behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and components.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Failure handling: mass

This page defines the failure handling for StaticBody3D and RigidBody3D, with particular attention to mass. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Mass is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sleeping is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for mass; distinguish required behavior from illustrative implementation detail.
- Keep sleeping behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and mass.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Diagnostics: sleeping

This page defines the diagnostics for StaticBody3D and RigidBody3D, with particular attention to sleeping. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Sleeping is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with CCD is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sleeping; distinguish required behavior from illustrative implementation detail.
- Keep CCD behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and sleeping.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Implementation sequence: CCD

This page defines the implementation sequence for StaticBody3D and RigidBody3D, with particular attention to CCD. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Ccd is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for CCD; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and CCD.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Verification: events

This page defines the verification for StaticBody3D and RigidBody3D, with particular attention to events. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with teleports is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep teleports behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Completion gate: teleports

This page defines the completion gate for StaticBody3D and RigidBody3D, with particular attention to teleports. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Teleports is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with components is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for teleports; distinguish required behavior from illustrative implementation detail.
- Keep components behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and teleports.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Purpose and boundary: components

This page defines the purpose and boundary for StaticBody3D and RigidBody3D, with particular attention to components. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Components is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with mass is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for components; distinguish required behavior from illustrative implementation detail.
- Keep mass behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and components.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Architecture: mass

This page defines the architecture for StaticBody3D and RigidBody3D, with particular attention to mass. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Mass is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sleeping is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for mass; distinguish required behavior from illustrative implementation detail.
- Keep sleeping behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and mass.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Data model: sleeping

This page defines the data model for StaticBody3D and RigidBody3D, with particular attention to sleeping. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Sleeping is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with CCD is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sleeping; distinguish required behavior from illustrative implementation detail.
- Keep CCD behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Staticbody3dAndRigidbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Staticbody3dAndRigidbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Staticbody3dAndRigidbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and sleeping.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.04 StaticBody3D and RigidBody3D

Public API: CCD

This page defines the public api for StaticBody3D and RigidBody3D, with particular attention to CCD. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Ccd is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for CCD; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Staticbody3d", "Staticbody3dAndRigidbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.04 and CCD.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Purpose and boundary: grounding

This page defines the purpose and boundary for CharacterBody3D and controller, with particular attention to grounding. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Grounding is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with steps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for grounding; distinguish required behavior from illustrative implementation detail.
- Keep steps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and grounding.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Architecture: steps

This page defines the architecture for CharacterBody3D and controller, with particular attention to steps. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Steps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with slopes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for steps; distinguish required behavior from illustrative implementation detail.
- Keep slopes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and steps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Data model: slopes

This page defines the data model for CharacterBody3D and controller, with particular attention to slopes. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Slopes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with moving platforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for slopes; distinguish required behavior from illustrative implementation detail.
- Keep moving platforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and slopes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Public API: moving platforms

This page defines the public api for CharacterBody3D and controller, with particular attention to moving platforms. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Moving platforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for moving platforms; distinguish required behavior from illustrative implementation detail.
- Keep network hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and moving platforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Ownership and lifetime: network hooks

This page defines the ownership and lifetime for CharacterBody3D and controller, with particular attention to network hooks. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Network hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with grounding is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for network hooks; distinguish required behavior from illustrative implementation detail.
- Keep grounding behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and network hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Threading model: grounding

This page defines the threading model for CharacterBody3D and controller, with particular attention to grounding. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Grounding is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with steps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for grounding; distinguish required behavior from illustrative implementation detail.
- Keep steps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and grounding.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Memory strategy: steps

This page defines the memory strategy for CharacterBody3D and controller, with particular attention to steps. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Steps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with slopes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for steps; distinguish required behavior from illustrative implementation detail.
- Keep slopes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and steps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Failure handling: slopes

This page defines the failure handling for CharacterBody3D and controller, with particular attention to slopes. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Slopes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with moving platforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for slopes; distinguish required behavior from illustrative implementation detail.
- Keep moving platforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and slopes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Diagnostics: moving platforms

This page defines the diagnostics for CharacterBody3D and controller, with particular attention to moving platforms. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Moving platforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for moving platforms; distinguish required behavior from illustrative implementation detail.
- Keep network hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and moving platforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Implementation sequence: network hooks

This page defines the implementation sequence for CharacterBody3D and controller, with particular attention to network hooks. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Network hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with grounding is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for network hooks; distinguish required behavior from illustrative implementation detail.
- Keep grounding behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and network hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Verification: grounding

This page defines the verification for CharacterBody3D and controller, with particular attention to grounding. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Grounding is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with steps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for grounding; distinguish required behavior from illustrative implementation detail.
- Keep steps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and grounding.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Completion gate: steps

This page defines the completion gate for CharacterBody3D and controller, with particular attention to steps. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Steps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with slopes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for steps; distinguish required behavior from illustrative implementation detail.
- Keep slopes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and steps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Purpose and boundary: slopes

This page defines the purpose and boundary for CharacterBody3D and controller, with particular attention to slopes. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Slopes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with moving platforms is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for slopes; distinguish required behavior from illustrative implementation detail.
- Keep moving platforms behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Characterbody3dAndControllerService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Characterbody3dAndControllerConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Characterbody3dAndControllerState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and slopes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.05 CharacterBody3D and controller

Architecture: moving platforms

This page defines the architecture for CharacterBody3D and controller, with particular attention to moving platforms. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Moving platforms is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with network hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for moving platforms; distinguish required behavior from illustrative implementation detail.
- Keep network hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Characterbody3d", "Characterbody3dAndController");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.05 and moving platforms.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Purpose and boundary: assets

This page defines the purpose and boundary for SoftBody3D, with particular attention to assets. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with constraints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assets; distinguish required behavior from illustrative implementation detail.
- Keep constraints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Architecture: constraints

This page defines the architecture for SoftBody3D, with particular attention to constraints. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Constraints is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with attachments is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for constraints; distinguish required behavior from illustrative implementation detail.
- Keep attachments behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and constraints.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Data model: attachments

This page defines the data model for SoftBody3D, with particular attention to attachments. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Attachments is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with render mesh coupling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for attachments; distinguish required behavior from illustrative implementation detail.
- Keep render mesh coupling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and attachments.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Public API: render mesh coupling

This page defines the public api for SoftBody3D, with particular attention to render mesh coupling. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Render mesh coupling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for render mesh coupling; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and render mesh coupling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Ownership and lifetime: budgets

This page defines the ownership and lifetime for SoftBody3D, with particular attention to budgets. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Threading model: assets

This page defines the threading model for SoftBody3D, with particular attention to assets. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with constraints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assets; distinguish required behavior from illustrative implementation detail.
- Keep constraints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Memory strategy: constraints

This page defines the memory strategy for SoftBody3D, with particular attention to constraints. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Constraints is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with attachments is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for constraints; distinguish required behavior from illustrative implementation detail.
- Keep attachments behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and constraints.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Failure handling: attachments

This page defines the failure handling for SoftBody3D, with particular attention to attachments. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Attachments is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with render mesh coupling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for attachments; distinguish required behavior from illustrative implementation detail.
- Keep render mesh coupling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and attachments.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Diagnostics: render mesh coupling

This page defines the diagnostics for SoftBody3D, with particular attention to render mesh coupling. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Render mesh coupling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for render mesh coupling; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and render mesh coupling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Implementation sequence: budgets

This page defines the implementation sequence for SoftBody3D, with particular attention to budgets. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Verification: assets

This page defines the verification for SoftBody3D, with particular attention to assets. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with constraints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assets; distinguish required behavior from illustrative implementation detail.
- Keep constraints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Completion gate: constraints

This page defines the completion gate for SoftBody3D, with particular attention to constraints. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Constraints is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with attachments is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for constraints; distinguish required behavior from illustrative implementation detail.
- Keep attachments behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and constraints.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Purpose and boundary: attachments

This page defines the purpose and boundary for SoftBody3D, with particular attention to attachments. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Attachments is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with render mesh coupling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for attachments; distinguish required behavior from illustrative implementation detail.
- Keep render mesh coupling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and attachments.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Architecture: render mesh coupling

This page defines the architecture for SoftBody3D, with particular attention to render mesh coupling. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Render mesh coupling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with budgets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for render mesh coupling; distinguish required behavior from illustrative implementation detail.
- Keep budgets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and render mesh coupling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Data model: budgets

This page defines the data model for SoftBody3D, with particular attention to budgets. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Budgets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with assets is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for budgets; distinguish required behavior from illustrative implementation detail.
- Keep assets behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class Softbody3dService final
{
public:
    [[nodiscard]] Result<void> Initialise(const Softbody3dConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    Softbody3dState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and budgets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.06 SoftBody3D

Public API: assets

This page defines the public api for SoftBody3D, with particular attention to assets. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Assets is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with constraints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for assets; distinguish required behavior from illustrative implementation detail.
- Keep constraints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Softbody3d", "Softbody3d");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.06 and assets.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Purpose and boundary: ray/shape casts

This page defines the purpose and boundary for Queries, triggers and constraints, with particular attention to ray/shape casts. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Ray/shape casts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with overlaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ray/shape casts; distinguish required behavior from illustrative implementation detail.
- Keep overlaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and ray/shape casts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Architecture: overlaps

This page defines the architecture for Queries, triggers and constraints, with particular attention to overlaps. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Overlaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for overlaps; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and overlaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Data model: events

This page defines the data model for Queries, triggers and constraints, with particular attention to events. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with joints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep joints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Public API: joints

This page defines the public api for Queries, triggers and constraints, with particular attention to joints. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Joints is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ragdolls is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for joints; distinguish required behavior from illustrative implementation detail.
- Keep ragdolls behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and joints.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Ownership and lifetime: ragdolls

This page defines the ownership and lifetime for Queries, triggers and constraints, with particular attention to ragdolls. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Ragdolls is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ray/shape casts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ragdolls; distinguish required behavior from illustrative implementation detail.
- Keep ray/shape casts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and ragdolls.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Threading model: ray/shape casts

This page defines the threading model for Queries, triggers and constraints, with particular attention to ray/shape casts. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Ray/shape casts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with overlaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ray/shape casts; distinguish required behavior from illustrative implementation detail.
- Keep overlaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and ray/shape casts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Memory strategy: overlaps

This page defines the memory strategy for Queries, triggers and constraints, with particular attention to overlaps. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Overlaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for overlaps; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and overlaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Failure handling: events

This page defines the failure handling for Queries, triggers and constraints, with particular attention to events. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with joints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep joints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Diagnostics: joints

This page defines the diagnostics for Queries, triggers and constraints, with particular attention to joints. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Joints is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ragdolls is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for joints; distinguish required behavior from illustrative implementation detail.
- Keep ragdolls behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and joints.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Implementation sequence: ragdolls

This page defines the implementation sequence for Queries, triggers and constraints, with particular attention to ragdolls. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Ragdolls is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ray/shape casts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ragdolls; distinguish required behavior from illustrative implementation detail.
- Keep ray/shape casts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and ragdolls.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Verification: ray/shape casts

This page defines the verification for Queries, triggers and constraints, with particular attention to ray/shape casts. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Ray/shape casts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with overlaps is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for ray/shape casts; distinguish required behavior from illustrative implementation detail.
- Keep overlaps behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and ray/shape casts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Completion gate: overlaps

This page defines the completion gate for Queries, triggers and constraints, with particular attention to overlaps. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Overlaps is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for overlaps; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and overlaps.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Purpose and boundary: events

This page defines the purpose and boundary for Queries, triggers and constraints, with particular attention to events. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with joints is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep joints behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class QueriesTriggersAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const QueriesTriggersAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    QueriesTriggersAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.07 Queries, triggers and constraints

Architecture: joints

This page defines the architecture for Queries, triggers and constraints, with particular attention to joints. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Joints is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with ragdolls is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for joints; distinguish required behavior from illustrative implementation detail.
- Keep ragdolls behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Queries", "QueriesTriggersAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.07 and joints.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Purpose and boundary: rebasing

This page defines the purpose and boundary for Large-world physics strategy prototype, with particular attention to rebasing. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Rebasing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with regions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rebasing; distinguish required behavior from illustrative implementation detail.
- Keep regions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LargeWorldPhysicsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LargeWorldPhysicsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LargeWorldPhysicsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and rebasing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Architecture: regions

This page defines the architecture for Large-world physics strategy prototype, with particular attention to regions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Regions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handoffs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for regions; distinguish required behavior from illustrative implementation detail.
- Keep handoffs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Large", "LargeWorldPhysics");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and regions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Data model: handoffs

This page defines the data model for Large-world physics strategy prototype, with particular attention to handoffs. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Handoffs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handoffs; distinguish required behavior from illustrative implementation detail.
- Keep precision tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LargeWorldPhysicsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LargeWorldPhysicsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LargeWorldPhysicsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and handoffs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Public API: precision tests

This page defines the public api for Large-world physics strategy prototype, with particular attention to precision tests. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Precision tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rebasing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision tests; distinguish required behavior from illustrative implementation detail.
- Keep rebasing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Large", "LargeWorldPhysics");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and precision tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Ownership and lifetime: rebasing

This page defines the ownership and lifetime for Large-world physics strategy prototype, with particular attention to rebasing. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Rebasing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with regions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rebasing; distinguish required behavior from illustrative implementation detail.
- Keep regions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LargeWorldPhysicsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LargeWorldPhysicsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LargeWorldPhysicsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and rebasing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Threading model: regions

This page defines the threading model for Large-world physics strategy prototype, with particular attention to regions. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Regions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handoffs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for regions; distinguish required behavior from illustrative implementation detail.
- Keep handoffs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Large", "LargeWorldPhysics");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and regions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Memory strategy: handoffs

This page defines the memory strategy for Large-world physics strategy prototype, with particular attention to handoffs. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Handoffs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handoffs; distinguish required behavior from illustrative implementation detail.
- Keep precision tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LargeWorldPhysicsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LargeWorldPhysicsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LargeWorldPhysicsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and handoffs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Failure handling: precision tests

This page defines the failure handling for Large-world physics strategy prototype, with particular attention to precision tests. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Precision tests is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rebasing is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for precision tests; distinguish required behavior from illustrative implementation detail.
- Keep rebasing behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Large", "LargeWorldPhysics");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and precision tests.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Diagnostics: rebasing

This page defines the diagnostics for Large-world physics strategy prototype, with particular attention to rebasing. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Rebasing is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with regions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rebasing; distinguish required behavior from illustrative implementation detail.
- Keep regions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LargeWorldPhysicsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LargeWorldPhysicsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LargeWorldPhysicsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and rebasing.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Implementation sequence: regions

This page defines the implementation sequence for Large-world physics strategy prototype, with particular attention to regions. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Regions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with handoffs is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for regions; distinguish required behavior from illustrative implementation detail.
- Keep handoffs behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Large", "LargeWorldPhysics");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and regions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.08 Large-world physics strategy prototype

Verification: handoffs

This page defines the verification for Large-world physics strategy prototype, with particular attention to handoffs. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Handoffs is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with precision tests is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for handoffs; distinguish required behavior from illustrative implementation detail.
- Keep precision tests behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LargeWorldPhysicsService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LargeWorldPhysicsConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LargeWorldPhysicsState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.08 and handoffs.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Purpose and boundary: poses

This page defines the purpose and boundary for Animation data and skeleton runtime, with particular attention to poses. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Poses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clips is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for poses; distinguish required behavior from illustrative implementation detail.
- Keep clips behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and poses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Architecture: clips

This page defines the architecture for Animation data and skeleton runtime, with particular attention to clips. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Clips is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compression is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clips; distinguish required behavior from illustrative implementation detail.
- Keep compression behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and clips.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Data model: compression

This page defines the data model for Animation data and skeleton runtime, with particular attention to compression. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Compression is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sampling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compression; distinguish required behavior from illustrative implementation detail.
- Keep sampling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and compression.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Public API: sampling

This page defines the public api for Animation data and skeleton runtime, with particular attention to sampling. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Sampling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with retargeting is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sampling; distinguish required behavior from illustrative implementation detail.
- Keep retargeting behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and sampling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Ownership and lifetime: retargeting

This page defines the ownership and lifetime for Animation data and skeleton runtime, with particular attention to retargeting. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Retargeting is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with poses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for retargeting; distinguish required behavior from illustrative implementation detail.
- Keep poses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and retargeting.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Threading model: poses

This page defines the threading model for Animation data and skeleton runtime, with particular attention to poses. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Poses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clips is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for poses; distinguish required behavior from illustrative implementation detail.
- Keep clips behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and poses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Memory strategy: clips

This page defines the memory strategy for Animation data and skeleton runtime, with particular attention to clips. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Clips is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compression is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clips; distinguish required behavior from illustrative implementation detail.
- Keep compression behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and clips.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Failure handling: compression

This page defines the failure handling for Animation data and skeleton runtime, with particular attention to compression. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Compression is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sampling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compression; distinguish required behavior from illustrative implementation detail.
- Keep sampling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and compression.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Diagnostics: sampling

This page defines the diagnostics for Animation data and skeleton runtime, with particular attention to sampling. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Sampling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with retargeting is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sampling; distinguish required behavior from illustrative implementation detail.
- Keep retargeting behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and sampling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Implementation sequence: retargeting

This page defines the implementation sequence for Animation data and skeleton runtime, with particular attention to retargeting. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Retargeting is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with poses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for retargeting; distinguish required behavior from illustrative implementation detail.
- Keep poses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and retargeting.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Verification: poses

This page defines the verification for Animation data and skeleton runtime, with particular attention to poses. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Poses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with clips is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for poses; distinguish required behavior from illustrative implementation detail.
- Keep clips behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and poses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Completion gate: clips

This page defines the completion gate for Animation data and skeleton runtime, with particular attention to clips. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Clips is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with compression is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for clips; distinguish required behavior from illustrative implementation detail.
- Keep compression behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and clips.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Purpose and boundary: compression

This page defines the purpose and boundary for Animation data and skeleton runtime, with particular attention to compression. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Compression is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with sampling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for compression; distinguish required behavior from illustrative implementation detail.
- Keep sampling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationDataAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationDataAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationDataAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and compression.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.09 Animation data and skeleton runtime

Architecture: sampling

This page defines the architecture for Animation data and skeleton runtime, with particular attention to sampling. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Sampling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with retargeting is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for sampling; distinguish required behavior from illustrative implementation detail.
- Keep retargeting behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationDataAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.09 and sampling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Purpose and boundary: blend trees

This page defines the purpose and boundary for Animation graphs and state machines, with particular attention to blend trees. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Blend trees is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with layers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for blend trees; distinguish required behavior from illustrative implementation detail.
- Keep layers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and blend trees.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Architecture: layers

This page defines the architecture for Animation graphs and state machines, with particular attention to layers. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Layers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with masks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for layers; distinguish required behavior from illustrative implementation detail.
- Keep masks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and layers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Data model: masks

This page defines the data model for Animation graphs and state machines, with particular attention to masks. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Masks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for masks; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and masks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Public API: events

This page defines the public api for Animation graphs and state machines, with particular attention to events. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root motion is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep root motion behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Ownership and lifetime: root motion

This page defines the ownership and lifetime for Animation graphs and state machines, with particular attention to root motion. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Root motion is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with IK is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for root motion; distinguish required behavior from illustrative implementation detail.
- Keep IK behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and root motion.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Threading model: IK

This page defines the threading model for Animation graphs and state machines, with particular attention to IK. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Ik is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with blend trees is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for IK; distinguish required behavior from illustrative implementation detail.
- Keep blend trees behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and IK.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Memory strategy: blend trees

This page defines the memory strategy for Animation graphs and state machines, with particular attention to blend trees. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Blend trees is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with layers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for blend trees; distinguish required behavior from illustrative implementation detail.
- Keep layers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and blend trees.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Failure handling: layers

This page defines the failure handling for Animation graphs and state machines, with particular attention to layers. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Layers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with masks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for layers; distinguish required behavior from illustrative implementation detail.
- Keep masks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and layers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Diagnostics: masks

This page defines the diagnostics for Animation graphs and state machines, with particular attention to masks. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Masks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for masks; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and masks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Implementation sequence: events

This page defines the implementation sequence for Animation graphs and state machines, with particular attention to events. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root motion is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep root motion behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Verification: root motion

This page defines the verification for Animation graphs and state machines, with particular attention to root motion. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Root motion is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with IK is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for root motion; distinguish required behavior from illustrative implementation detail.
- Keep IK behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and root motion.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Completion gate: IK

This page defines the completion gate for Animation graphs and state machines, with particular attention to IK. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Ik is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with blend trees is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for IK; distinguish required behavior from illustrative implementation detail.
- Keep blend trees behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and IK.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Purpose and boundary: blend trees

This page defines the purpose and boundary for Animation graphs and state machines, with particular attention to blend trees. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Blend trees is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with layers is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for blend trees; distinguish required behavior from illustrative implementation detail.
- Keep layers behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and blend trees.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Architecture: layers

This page defines the architecture for Animation graphs and state machines, with particular attention to layers. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Layers is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with masks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for layers; distinguish required behavior from illustrative implementation detail.
- Keep masks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and layers.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Data model: masks

This page defines the data model for Animation graphs and state machines, with particular attention to masks. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Masks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with events is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for masks; distinguish required behavior from illustrative implementation detail.
- Keep events behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AnimationGraphsAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AnimationGraphsAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AnimationGraphsAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and masks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.10 Animation graphs and state machines

Public API: events

This page defines the public api for Animation graphs and state machines, with particular attention to events. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Events is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with root motion is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for events; distinguish required behavior from illustrative implementation detail.
- Keep root motion behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Animation", "AnimationGraphsAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.10 and events.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Purpose and boundary: backend

This page defines the purpose and boundary for Audio architecture, with particular attention to backend. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Backend is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with voices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend; distinguish required behavior from illustrative implementation detail.
- Keep voices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and backend.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Architecture: voices

This page defines the architecture for Audio architecture, with particular attention to voices. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Voices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with buses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for voices; distinguish required behavior from illustrative implementation detail.
- Keep buses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Audio", "AudioArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and voices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Data model: buses

This page defines the data model for Audio architecture, with particular attention to buses. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Buses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with 3D spatialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for buses; distinguish required behavior from illustrative implementation detail.
- Keep 3D spatialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and buses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Public API: 3D spatialization

This page defines the public api for Audio architecture, with particular attention to 3D spatialization. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

3d spatialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with streaming is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for 3D spatialization; distinguish required behavior from illustrative implementation detail.
- Keep streaming behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Audio", "AudioArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and 3D spatialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Ownership and lifetime: streaming

This page defines the ownership and lifetime for Audio architecture, with particular attention to streaming. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Streaming is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with large worlds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for streaming; distinguish required behavior from illustrative implementation detail.
- Keep large worlds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and streaming.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Threading model: large worlds

This page defines the threading model for Audio architecture, with particular attention to large worlds. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Large worlds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for large worlds; distinguish required behavior from illustrative implementation detail.
- Keep backend behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Audio", "AudioArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and large worlds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Memory strategy: backend

This page defines the memory strategy for Audio architecture, with particular attention to backend. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Backend is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with voices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend; distinguish required behavior from illustrative implementation detail.
- Keep voices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and backend.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Failure handling: voices

This page defines the failure handling for Audio architecture, with particular attention to voices. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Voices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with buses is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for voices; distinguish required behavior from illustrative implementation detail.
- Keep buses behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Audio", "AudioArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and voices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Diagnostics: buses

This page defines the diagnostics for Audio architecture, with particular attention to buses. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Buses is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with 3D spatialization is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for buses; distinguish required behavior from illustrative implementation detail.
- Keep 3D spatialization behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and buses.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Implementation sequence: 3D spatialization

This page defines the implementation sequence for Audio architecture, with particular attention to 3D spatialization. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

3d spatialization is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with streaming is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for 3D spatialization; distinguish required behavior from illustrative implementation detail.
- Keep streaming behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Audio", "AudioArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and 3D spatialization.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Verification: streaming

This page defines the verification for Audio architecture, with particular attention to streaming. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Streaming is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with large worlds is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for streaming; distinguish required behavior from illustrative implementation detail.
- Keep large worlds behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and streaming.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Completion gate: large worlds

This page defines the completion gate for Audio architecture, with particular attention to large worlds. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Large worlds is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with backend is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for large worlds; distinguish required behavior from illustrative implementation detail.
- Keep backend behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Audio", "AudioArchitecture");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and large worlds.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.11 Audio architecture

Purpose and boundary: backend

This page defines the purpose and boundary for Audio architecture, with particular attention to backend. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Backend is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with voices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for backend; distinguish required behavior from illustrative implementation detail.
- Keep voices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class AudioArchitectureService final
{
public:
    [[nodiscard]] Result<void> Initialise(const AudioArchitectureConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    AudioArchitectureState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.11 and backend.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Purpose and boundary: devices

This page defines the purpose and boundary for Input system, with particular attention to devices. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Devices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with actions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for devices; distinguish required behavior from illustrative implementation detail.
- Keep actions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class InputSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const InputSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    InputSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and devices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Architecture: actions

This page defines the architecture for Input system, with particular attention to actions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Actions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with contexts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for actions; distinguish required behavior from illustrative implementation detail.
- Keep contexts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Input", "InputSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and actions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Data model: contexts

This page defines the data model for Input system, with particular attention to contexts. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Contexts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rebinding is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for contexts; distinguish required behavior from illustrative implementation detail.
- Keep rebinding behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class InputSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const InputSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    InputSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and contexts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Public API: rebinding

This page defines the public api for Input system, with particular attention to rebinding. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Rebinding is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with gamepad is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rebinding; distinguish required behavior from illustrative implementation detail.
- Keep gamepad behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Input", "InputSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and rebinding.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Ownership and lifetime: gamepad

This page defines the ownership and lifetime for Input system, with particular attention to gamepad. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Gamepad is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with editor separation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for gamepad; distinguish required behavior from illustrative implementation detail.
- Keep editor separation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class InputSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const InputSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    InputSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and gamepad.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Threading model: editor separation

This page defines the threading model for Input system, with particular attention to editor separation. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Editor separation is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with devices is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for editor separation; distinguish required behavior from illustrative implementation detail.
- Keep devices behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Input", "InputSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and editor separation.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Memory strategy: devices

This page defines the memory strategy for Input system, with particular attention to devices. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Devices is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with actions is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for devices; distinguish required behavior from illustrative implementation detail.
- Keep actions behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class InputSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const InputSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    InputSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and devices.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Failure handling: actions

This page defines the failure handling for Input system, with particular attention to actions. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Actions is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with contexts is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for actions; distinguish required behavior from illustrative implementation detail.
- Keep contexts behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Input", "InputSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and actions.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Diagnostics: contexts

This page defines the diagnostics for Input system, with particular attention to contexts. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Contexts is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with rebinding is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for contexts; distinguish required behavior from illustrative implementation detail.
- Keep rebinding behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class InputSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const InputSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    InputSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and contexts.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Implementation sequence: rebinding

This page defines the implementation sequence for Input system, with particular attention to rebinding. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Rebinding is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with gamepad is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for rebinding; distinguish required behavior from illustrative implementation detail.
- Keep gamepad behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Input", "InputSystem");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and rebinding.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.12 Input system

Verification: gamepad

This page defines the verification for Input system, with particular attention to gamepad. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Gamepad is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with editor separation is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for gamepad; distinguish required behavior from illustrative implementation detail.
- Keep editor separation behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class InputSystemService final
{
public:
    [[nodiscard]] Result<void> Initialise(const InputSystemConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    InputSystemState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.12 and gamepad.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Purpose and boundary: baking

This page defines the purpose and boundary for Navigation mesh and agents, with particular attention to baking. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Baking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tiles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for baking; distinguish required behavior from illustrative implementation detail.
- Keep tiles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and baking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Architecture: tiles

This page defines the architecture for Navigation mesh and agents, with particular attention to tiles. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Tiles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tiles; distinguish required behavior from illustrative implementation detail.
- Keep queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Navigation", "NavigationMeshAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and tiles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Data model: queries

This page defines the data model for Navigation mesh and agents, with particular attention to queries. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with obstacles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for queries; distinguish required behavior from illustrative implementation detail.
- Keep obstacles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Public API: obstacles

This page defines the public api for Navigation mesh and agents, with particular attention to obstacles. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Obstacles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with streaming is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for obstacles; distinguish required behavior from illustrative implementation detail.
- Keep streaming behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Navigation", "NavigationMeshAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and obstacles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Ownership and lifetime: streaming

This page defines the ownership and lifetime for Navigation mesh and agents, with particular attention to streaming. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Streaming is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with baking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for streaming; distinguish required behavior from illustrative implementation detail.
- Keep baking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and streaming.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Threading model: baking

This page defines the threading model for Navigation mesh and agents, with particular attention to baking. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Baking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tiles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for baking; distinguish required behavior from illustrative implementation detail.
- Keep tiles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Navigation", "NavigationMeshAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and baking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Memory strategy: tiles

This page defines the memory strategy for Navigation mesh and agents, with particular attention to tiles. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Tiles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tiles; distinguish required behavior from illustrative implementation detail.
- Keep queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and tiles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Failure handling: queries

This page defines the failure handling for Navigation mesh and agents, with particular attention to queries. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract defines validation, recoverable errors, fatal conditions, rollback, and degraded operation.

Queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to reject its inputs, establish state transitions, and validate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with obstacles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for queries; distinguish required behavior from illustrative implementation detail.
- Keep obstacles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Navigation", "NavigationMeshAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Diagnostics: obstacles

This page defines the diagnostics for Navigation mesh and agents, with particular attention to obstacles. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract identifies logs, counters, trace events, debug views, and evidence needed to understand behavior.

Obstacles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to instrument its inputs, enforce state transitions, and isolate failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with streaming is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for obstacles; distinguish required behavior from illustrative implementation detail.
- Keep streaming behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and obstacles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Implementation sequence: streaming

This page defines the implementation sequence for Navigation mesh and agents, with particular attention to streaming. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract orders the work so each step produces a testable vertical slice without premature abstraction.

Streaming is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to document its inputs, record state transitions, and measure failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with baking is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for streaming; distinguish required behavior from illustrative implementation detail.
- Keep baking behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Navigation", "NavigationMeshAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and streaming.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Verification: baking

This page defines the verification for Navigation mesh and agents, with particular attention to baking. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract defines unit, integration, stress, performance, and cross-compiler evidence.

Baking is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to establish its inputs, validate state transitions, and bound failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with tiles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for baking; distinguish required behavior from illustrative implementation detail.
- Keep tiles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and baking.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Completion gate: tiles

This page defines the completion gate for Navigation mesh and agents, with particular attention to tiles. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract lists the objective criteria that permit dependent chapters to begin.

Tiles is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to enforce its inputs, isolate state transitions, and expose failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for tiles; distinguish required behavior from illustrative implementation detail.
- Keep queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Navigation", "NavigationMeshAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and tiles.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.13 Navigation mesh and agents

Purpose and boundary: queries

This page defines the purpose and boundary for Navigation mesh and agents, with particular attention to queries. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with obstacles is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for queries; distinguish required behavior from illustrative implementation detail.
- Keep obstacles behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class NavigationMeshAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const NavigationMeshAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    NavigationMeshAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.13 and queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Purpose and boundary: blackboards

This page defines the purpose and boundary for Lightweight AI services, with particular attention to blackboards. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Blackboards is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with state machines is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for blackboards; distinguish required behavior from illustrative implementation detail.
- Keep state machines behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LightweightServicesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LightweightServicesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LightweightServicesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and blackboards.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Architecture: state machines

This page defines the architecture for Lightweight AI services, with particular attention to state machines. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

State machines is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with perception hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for state machines; distinguish required behavior from illustrative implementation detail.
- Keep perception hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Lightweight", "LightweightServices");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and state machines.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Data model: perception hooks

This page defines the data model for Lightweight AI services, with particular attention to perception hooks. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Perception hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with spatial queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for perception hooks; distinguish required behavior from illustrative implementation detail.
- Keep spatial queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LightweightServicesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LightweightServicesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LightweightServicesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and perception hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Public API: spatial queries

This page defines the public api for Lightweight AI services, with particular attention to spatial queries. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Spatial queries is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with blackboards is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for spatial queries; distinguish required behavior from illustrative implementation detail.
- Keep blackboards behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Lightweight", "LightweightServices");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and spatial queries.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Ownership and lifetime: blackboards

This page defines the ownership and lifetime for Lightweight AI services, with particular attention to blackboards. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Blackboards is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with state machines is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for blackboards; distinguish required behavior from illustrative implementation detail.
- Keep state machines behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LightweightServicesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LightweightServicesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LightweightServicesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and blackboards.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Threading model: state machines

This page defines the threading model for Lightweight AI services, with particular attention to state machines. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

State machines is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with perception hooks is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for state machines; distinguish required behavior from illustrative implementation detail.
- Keep perception hooks behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Lightweight", "LightweightServices");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and state machines.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.14 Lightweight AI services

Memory strategy: perception hooks

This page defines the memory strategy for Lightweight AI services, with particular attention to perception hooks. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Perception hooks is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with spatial queries is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for perception hooks; distinguish required behavior from illustrative implementation detail.
- Keep spatial queries behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class LightweightServicesService final
{
public:
    [[nodiscard]] Result<void> Initialise(const LightweightServicesConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    LightweightServicesState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.14 and perception hooks.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Purpose and boundary: stress scenes

This page defines the purpose and boundary for Simulation test and performance gate, with particular attention to stress scenes. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract defines what the subsystem owns, what it deliberately excludes, and the observable contract consumed by dependent modules.

Stress scenes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to record its inputs, measure state transitions, and preserve failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for stress scenes; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationTestAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationTestAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationTestAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and stress scenes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Architecture: determinism

This page defines the architecture for Simulation test and performance gate, with particular attention to determinism. The implementation must be thread-safe, portable, and independently verifiable under MSVC and clang-cl. The contract maps responsibilities into modules, services, data flows, and explicit dependency edges.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to validate its inputs, bound state transitions, and reject failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with latency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep latency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationTestAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Data model: latency

This page defines the data model for Simulation test and performance gate, with particular attention to latency. The implementation must be diagnosable, bounded, and independently verifiable under MSVC and clang-cl. The contract specifies the stable types, identifiers, descriptors, records, and state transitions required by the implementation.

Latency is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to isolate its inputs, expose state transitions, and instrument failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with memory is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for latency; distinguish required behavior from illustrative implementation detail.
- Keep memory behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationTestAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationTestAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationTestAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and latency.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Public API: memory

This page defines the public api for Simulation test and performance gate, with particular attention to memory. The implementation must be testable, cache-conscious, and independently verifiable under MSVC and clang-cl. The contract defines caller-visible functions, result semantics, invariants, and versioning expectations.

Memory is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to measure its inputs, preserve state transitions, and document failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with profiling is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for memory; distinguish required behavior from illustrative implementation detail.
- Keep profiling behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationTestAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and memory.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Ownership and lifetime: profiling

This page defines the ownership and lifetime for Simulation test and performance gate, with particular attention to profiling. The implementation must be recoverable, deterministic, and independently verifiable under MSVC and clang-cl. The contract assigns creation, retention, transfer, invalidation, and destruction responsibilities.

Profiling is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to bound its inputs, reject state transitions, and establish failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with stress scenes is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for profiling; distinguish required behavior from illustrative implementation detail.
- Keep stress scenes behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationTestAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationTestAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationTestAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and profiling.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Threading model: stress scenes

This page defines the threading model for Simulation test and performance gate, with particular attention to stress scenes. The implementation must be plugin-safe, versioned, and independently verifiable under MSVC and clang-cl. The contract defines thread affinity, parallel work, synchronization points, cancellation, and shutdown behavior.

Stress scenes is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to expose its inputs, instrument state transitions, and enforce failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with determinism is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for stress scenes; distinguish required behavior from illustrative implementation detail.
- Keep determinism behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
SKEIN_TRACE_SCOPE("Simulation", "SimulationTestAnd");
SKEIN_ASSERT(IsOnExpectedThread());
auto result = Validate(description);
if (!result)
    return Unexpected(result.Error());
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and stress scenes.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.

07.15 Simulation test and performance gate

Memory strategy: determinism

This page defines the memory strategy for Simulation test and performance gate, with particular attention to determinism. The implementation must be explicit, allocation-aware, and independently verifiable under MSVC and clang-cl. The contract sets allocation domains, budgets, transient storage, pooling, and diagnostic tags.

Determinism is treated as an owned engineering concern rather than an incidental detail. The chapter requires the implementation to preserve its inputs, document state transitions, and record failure evidence. No caller may depend on private storage layout, backend-specific handles, undocumented thread behavior, or allocator side effects.

The primary interaction with latency is expressed through stable descriptors and handles. Dependent modules consume immutable descriptions or narrow service interfaces; they do not reach into internal containers. All externally visible identifiers remain valid only for their documented generation and lifetime, and stale references must fail predictably in development builds.

Implementation work should begin with the smallest executable path that proves the contract: configuration, construction, one successful operation, one deliberate failure, shutdown, and retained diagnostics. Optimizations follow only after the baseline is covered by tests and SkeinTrace markers. Performance changes must preserve behavior and include before/after evidence.

DESIGN OBLIGATIONS

- Define the normative contract for determinism; distinguish required behavior from illustrative implementation detail.
- Keep latency behind SKEIN-owned interfaces so third-party or platform code cannot leak into callers.
- Assign each mutable field to one owning thread or protect it with a documented synchronization primitive.
- Tag persistent, transient, frame, and scratch allocations separately; expose budgets through diagnostics.

```
class SimulationTestAndService final
{
public:
    [[nodiscard]] Result<void> Initialise(const SimulationTestAndConfig& config);
    void Tick(const FrameContext& frame);
    void Shutdown() noexcept;
private:
    SimulationTestAndState m_state{};
};
```

REQUIRED EVIDENCE

- A focused test named for 07.15 and determinism.
- SkeinTrace events showing startup, operation, failure, and shutdown.
- Allocation and thread-affinity assertions enabled in Debug.
- Build and test results for msvc-debug, msvc-release, clang-debug, and clang-release.